AlphaMathComp

Tristan Lambert
Inria
ENS Rennes

Jules Viennot
Université Paris Cité
IRIF, Inria, CNRS

Guillaume Baudart
Université Paris Cité
IRIF, Inria, CNRS

Marc Lelarge
DI ENS, PSL University
Inria

***Abstract*—Large language models have recently demonstrated promising capabilities for automated formal theorem proving, but generating correct proofs remains challenging, particularly in specialized proof environments such as the Mathematical Components library. In this work, we investigate whether verifier feedback can be used to improve language-model-based proof search in this setting. We adapt Treefinement, a proof refinement method originally introduced for formally verified full program, annotations and proofs generation in Verus, to tactic-by-tactic proof construction in Rocq. Rather than generating complete proofs, our adaptation of Treefinement consists of building a search tree whose nodes correspond to individual tactic attempts and using the feedback returned by the proof assistant to retain useful correction examples across successive attempts. We compare this approach to naive full-proof and tactic-by-tactic generations, and several Treefinement variants. Our experiments show that our tactic-by-tactic Treefinement achieves better results than the considered baselines when using suitable search parameters.**[1]

1. Introduction

Large Language Models (LLMs) are becomming increasingly capable of solving mathematical problems, recently showing ability to overcome problems that even researchers didn't manage to solve, both with and without the help of humans [4] [3]. Such improvements reinforce the idea that LLMs may become fully autonomous in the process of solving mathematical problems in the future. Nevertheless, even if it becomes true, these models are going to increase the number of attempts to solving problems and they can make mistakes, so reviewing their mathematical production is a necessary but increasingly difficult task. To address this issue, the most promising solution is certainly to use formal proof assistants, such as Isabelle [5], Lean [6] or Rocq [7]. They provide a framework in which mathematical proofs can be mechanically verified with the highest degree of certainty. The natural next step is therefore to ask language models to generate formal proofs directly in these languages. Figure 1 shows an example of a proof in Mathematical Component, the Rocq library, which we will focus on in this paper.

This objective has recently become considerably more realistic thanks to the rapid progress of large language models. General-purpose language models have demonstrated the ability to generate formal proofs in interactive theorem provers, often with little task-specific training [8]. It seems interesting to use the feedback returned by the proof assistant not only to reject invalid proof attempts, but also to guide future generations. It has been shown that compiler or type-checking errors can significantly improve the quality of subsequent generations by enabling iterative refinement and self-correction [9], [10]. However, these approaches typically exploit this feedback only locally, correcting the current attempt before continuing the search, while the information collected during unsuccessful explorations is discarded once a proof has been found or the search terminates. On the other hand, AlphaVerus tries to leverage feedback globally across multiple attempts [1], and to do so introduced the Treefinement, a tree search algorithm for proof refinement using verifier feedback.

Treefinement was originally introduced in AlphaVerus for formally verified code generation in Verus, which is a verification framework for Rust programs: the objective is to generate executable code together with specifications and proof annotations establishing its correctness. AlphaVerus therefore oper-

1. The code is available on GitHub : https://github.com/Triscoma/alphaMathComp



Figure 1. Example of a proof in Mathematical Components. A proof consists of a succession of commands called *tactics*. When writing a proof, the logical correctness of the tactics we write is verified in order (this verification is called *type-checking*) and we have direct access to various information regarding its progress, notably the evolution of hypotheses and the current goal.

ates in a programming-oriented setting. In contrast, MathComp is a library for formalized mathematics in Rocq, where the main objective is to construct mathematical proofs rather than to verify the functional correctness of generated programs. This difference also leads to different proof-generation challenges. In Verus, proofs may involve reasoning about program states, specifications, loop invariants, and properties of executable code, whereas MathComp is built upon the ssreflect tactic language and on the mathematical structures and lemmas provided by the library. Both environments are challenging for language models because of the limited availability of training data for their respective formal languages and proof idioms, but the knowledge required to navigate them may be of a different nature. We therefore want to study whether the Treefinement idea developed for verified program generation can also be effective in a conceptually different, mathematics-oriented formalization environment where, notably, reasoning step-by-step may be more efficient than directly generating full-proofs.

The main contributions of this work are the following:

— We adapt Treefinement to the setting of tactic-by-tactic proof generation in MathComp, where each node of the search tree corresponds to a single proof step rather than an entire proof.

— We evaluate the impact of this adaptation through an ablation study, comparing it with full-proof generation and other baselines that isolate the effects of the different parts of our adaptation of Treefinement.

2. Related work

LLMs for Formal Theorem Proving

LLMs in formal theorem proving are used in combination with symbolic proof assistants to address formal mathematical tasks. Two primary methodologies for LLMs formal theorem proving have emerged: single-pass methods and proof-step methods. Single-pass methods such as DSP [13] aim to generate entire proofs at once using prompts enriched with contextual information. Step provers such as GPT-f [14] progressively make progress toward a given goal by asking the LLM to find the next best tactic to apply, conditioned on the current state of the proof environment. These methods are generally combined with tree search to efficiently build the proof upon the most promising branches.

Verifier-Guided Generation and Feedback-Based Refinement

It has been shown that textual feedback on language models outputs can improve their subsequent generations [9] [10]. The same idea can be transferred to theorem proving using the output of the verifier combined with previous examples of successful proof steps as a feedback to make the model refine its answers [15], [16], [17]. Furthermore, AlphaVerus [1] showed that accumulating information across multiple tries leads to better results than simply exploiting verifier feedback to repair locally the current generation. AlphaVerus introduced Treefinement, a proof refinement method consisting of a tree search which memorizes the good proof and correction steps found by the model during the exploration.

3. Method

3.1. Treefinement

Given a goal in MathComp, we aim to have the LLM identify the best candidate for the next tactic needed to solve it. To achieve this, we store its best proposals throughout the successive attempts made by the model, so that we can later present these successful attempts back to it, hoping that this

additional information will improve its accuracy.

To do so, we use the Treefinement method. During proof generation, we build a tree where the nodes represents a proof state, and the edges going from such a node corresponds to an attempt made by the model to continue the proof from that state. At each iteration we consider the most promising node of the tree and ask the LLM to try to correct the errors that appear in this node. When the model manages to complete the proof, we save all the transitions leading to the successful node, as examples of good correction steps we will use later to feed the LLM.

3.2. Implementation

At each iteration, we process some leaves of the current tree using a beam search. We first select the most promising nodes according to a given criterion, and then there are three possible cases:

1. The node corresponds to a list of goals, possibly together with feedback obtained from commands such as Check or Search.

2. The node corresponds to a list of goals together with a previous attempt that failed to type-check.

3. The node corresponds to an empty list of goals, meaning that the model has successfully completed the proof.

In case 1, we simply prompt the LLM with the first of the current goals and any available feedback from the type-checker to generate some number of new nodes

In case 2, we prompt the LLM with the first of the current goals, the previous command it attempted, the corresponding type-checking error, and any additional feedback that may have been collected during earlier interactions, and we generate some number of new nodes from this.

In case 3, starting from the successful node, we repeatedly extract all the useful correction examples that can be inferred from the tree structure. This includes every pair of consecutive nodes on the successful path from the root to the successful node, where the child corresponds to a tactic that type-checked. Moreover, let's consider the pairs of nodes $(n_{\text{err}}, n_{\text{good}})$ such that n_{good} lies on the successful branch and corresponds to a tactic that type-checks, while n_{err} belongs to the subtree rooted in the first ancestor of n_{good} and induced by the nodes corresponding to tactics which didn't type-check (see Figure 2). Since proof is generated tactic by tactic, the goals in the proof state of these two nodes are the same, so n_{good} can be seen as a correction of the node n_{err}. The "ideas" behind the tactics used in these nodes might differ, but it would anyway concern only one tactic of the final proof we generate so we decided to extract them as well.

The set of saved transitions is then randomly sampled at the beginning of each iteration and used as example to help the model answering our future queries. Initially, instead of starting with an empty set we insert a few examples of correct proof steps in the set to guide the model on its very first attempts.

We implemented it in python, using the Pytanque API [18] to communicate with the Rocq proof assistant via coq-lsp and made it run on the Jean Zay supercomputer. For better execution time we execute the Treefinement in parallel on many instances, and also explore the $\mathrm{beam_size}$ best nodes in parallel at each iteration of Treefinement inside each tree. process each branch in parallel The pseudo-code of our Treefinement can be found as Algorithm 1 and Algorithm 2 in the appendix.

4. Experimental Setup

4.1. Parameters

We see from the description of the algorithm that we still have some choice on parameters :

- $\mathbf{arity}$, the number of new nodes we create each time we explore a leaf, leading to a trade-off between broad and deep exploration. Since we type-check each tactic generated, and an higher arity implies more tactic generated, we expect an higher arity to lead to better results for the easiest problems. However, for more difficult problems a deeper exploration

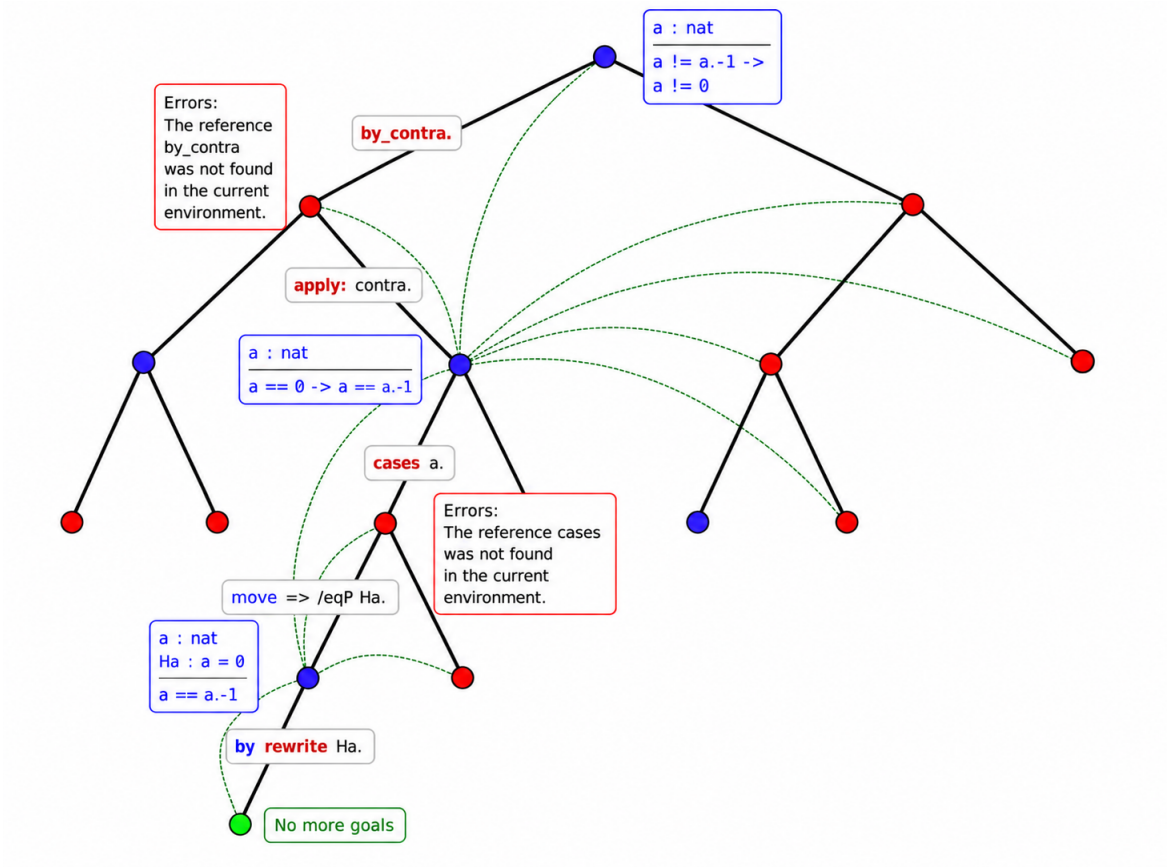


Figure 2. Example of Treefinement that could have produced the proof in Figure 1. The dotted lines indicate the saved transitions. The proof state corresponding to the complete proof is shown in green, other proof states obtained after a tactic type-checked are shown in blue, and proof states corresponding to a tactic that failed to type-check are shown in red. We annotated the main branch with an example of tactics and proof states we could have had.

may be required and thus a small arity is expected to be more likely to achieve the best results.

- **iter_max**, the number of leaves we are going to explore before giving up on the proof. The higher this parameter is the better are the results going to be. Yet the best choice for the other parameters may depend of the value of iter_max.

- **penalty**, the heuristic we use to guess if leaves are promising or not. We will only use

$$\frac{\text{number of errors in the branch}}{\text{number of nodes in the branch}}.$$

- **number_examples**, the number of examples among those we already collected we want to choose to feed the LLM when we explore a leaf.

We will then try to determine through experiments what are the best combinations of parameters to achieve the best results.

4.2. Baselines

Our goal is to generate formally verified proofs and our method can be separated into 3 different layers which are : generate proofs tactic by tactic , taking previous errors and type-checker feedback into account, and using Treefinement to do so.
To evaluate the performance of our approach we will compare the results of these different components

difficulty	theorems	tactics
1	49	131
2	22	82
3	20	60
4	10	40
5	7	30
6	5	23
7	1	18
8	3	17
9	3	14
10	1	11
11	1	10
12+	9	60
total	131	496
average	2.7	4.3

TABLE 1. THEOREMS AND TACTICS DISTRIBUTION OF THE DATASET

taken independently, in both full-proof and step-by-step approaches :

1) Naive generation
2) Treefinement with arity of 1
3) Treefinement without access to previous examples
4) Complete Treefinement

4.3. Dataset

As a dataset, we use the 2022 MathComp School exercises [2] which we slightly modified so that it can type-check in our version of MathComp. To artificially augment the size of the dataset, we use, in addition to the theorems we have, each goal produced after applying a tactic present in the correction as a statement to prove. The Table 1 and the Figure 2 show the theorems and the tactics in the final dataset, ranked by difficulty, ie. the number of tactics separating them to the end of the proof in the correction.

Since the algorithm is supposed to make progress throughout its execution, we will apply Treefinement to our statements in increasing order of difficulty, and because different statements of our dataset can be two goals appearing at two different times in a same proof of the dataset, we exclude the collected data from all statements appearing in the proof of the same theorem as the one being examined, when selecting repair examples to feed the LLM.

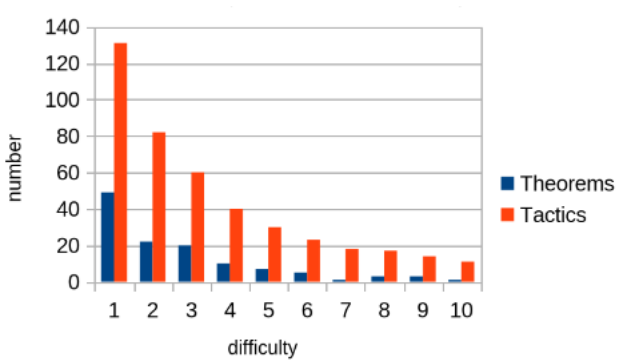


Figure 3. Theorems and tactics distribution of the dataset

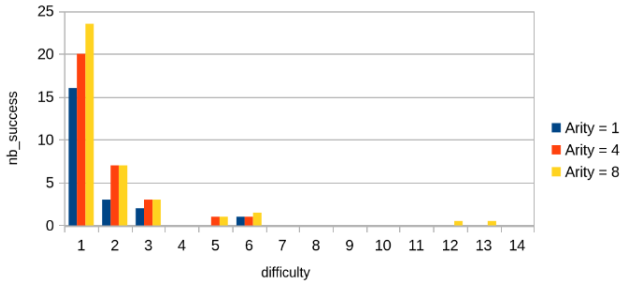


Figure 4. Results in function of arity

5. Results

We use the model Qwen3-8b for the experiments.

We first try to find the best parameters for our method. We fix iter_max to 15 and beam_size to 4, and start with number_examples $= 0$. We then try different values of arity to determine the best one. We tested for 1, 4 and 8. Figure 3 shows these results. The best performances are reached for arity $= 8$, so we stick to that value for the following experiments.

After having decided what is the best arity among the values we tried, we search the best value for number_examples. We try 0, 5, 20 and 50. The results are shown in Figure 4. Although the results appear to be better as soon as number_examples > 0 the exact value doesn't impact much the results, but 20 seems to be the best choice.

Finally, we can compare the results with the naive full proof generation and other versions of Treefinement for full proof, closer to what we find in AlphaVerus. Results are shown in Figure 5. We chose to

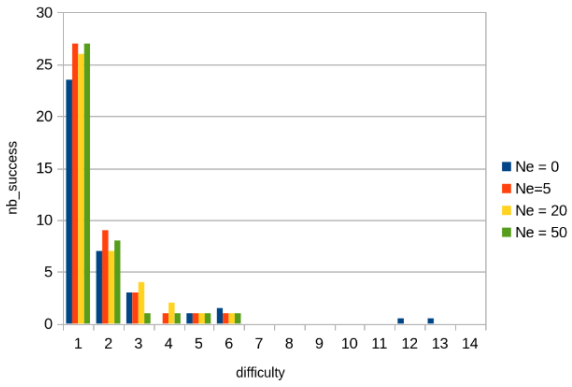


Figure 5. Results in function of number_examples (Ne)

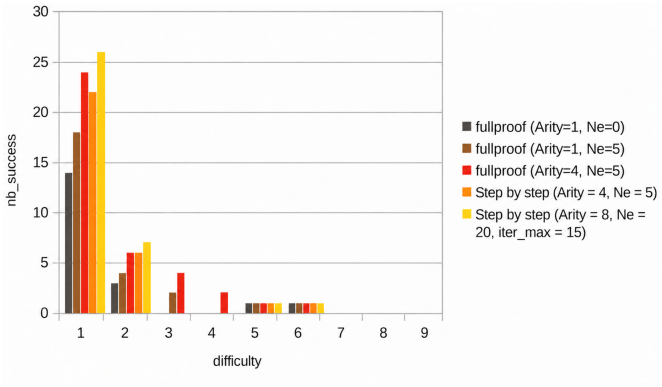


Figure 6. Comparison with fullproof baselines, iter_max $= 10$

made these experiments with an arity slightly lower, since generating a whole proof requires more tokens, and we didn't tried with the largest values of arity and number_examples for the same reason. We observe that, as AlphaVerus showed, using Treefinement to generate full proofs increases the performance; and the results in step-by-step with similar parameters are essentially the same. To compensate that it is less costly to generate only one tactic than a full proof, we can also compare it with step-by-step treefinement with slighly better parameters. When we perform step-by-step Treefinement with the parameters we found previously, and with an iter_max of 15, the achieved performances are higher.

6. Conclusion

In this work, we investigated the use of verifier feedback to improve automated tactic-by-tactic theorem proving in the MathComp library. We adapted Treefinement, originally introduced for refining complete proofs, to a setting in which each node of the search tree represents a single tactic applied to the current proof state. This formulation allows the system not only to exploit successful proof steps, but also to retain correction examples extracted from failed attempts, thereby accumulating useful information throughout the search.

Our experiments with Qwen3-8B on the MathComp School dataset suggest that this approach is effective. In particular, the tactic-by-tactic formulation benefits from generating several candidate tactics at each step, the comparison with the different baselines indicates that step-by-step Treefinement achieves better results than both step-by-step generation and full-proof Treefinement, at least in specialized formal environments such as MathComp, where the limited representation in training data makes direct proof generation particularly challenging.

To deepen this work, further experiments on larger and more diverse datasets, with other language models and broader parameter searches, could be interesting to establish how well the observed improvements generalize. A possible direction for future work would also be to investigate more sophisticated criteria for selecting promising nodes and saved examples, as well as methods for using the accumulated feedback more efficiently.

Overall, our results indicate that combining step-by-step proof generation with a tree searach and a global memory of verifier feedback is a promising approach to automated formal theorem proving. This provides a natural framework for further improving the ability of language models to construct formally verified proofs in MathComp and other similar theorem-proving environments.

References

[1] Pranjal Aggarwal, Bryan Parno and Sean Welleck, *AlphaVerus: Bootstrapping Formally Verified Code Generation through Self-Improving Translation and Treefinement*, arXiv:2412.06176 [cs.LG], 2024.

[2] Assia Mahboubi and Enrico Tassi, *2022 MathComp School*, https://mathcomp-schools.gitlabpages.inria.fr/2022-12-school/

[3] OpenAI, *An OpenAI model has disproved a central conjecture in discrete geometry*, https://openai.com/index/model-disproves-discrete-geometry-conjecture/, 2026

[4] Edgar Dobriban *Solving a Research Problem in Mathematical Statistics with AI Assistance*, arXiv:2511.18828 [math.ST], 2025

[5] Isabelle. Isabelle. https://isabelle.in.tum.de/

[6] Lean FRO, Lean Theorem Prover. https://lean-lang.org/

[7] Rocq Development Team, The Rocq Prover. https://rocq-prover.org/

[8] Kaiyu Yang, Aidan Nitenson, Yilun Han, et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, Advances in Neural Information Processing Systems (NeurIPS), 2023.

[9] Aman Madaan, Niket Tandon, Prakhar Gupta, et al., *Self-Refine: Iterative Refinement with Self-Feedback*, Advances in Neural Information Processing Systems (NeurIPS), 2023.

[10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan and Shunyu Yao, *Reflexion: Language Agents with Verbal Reinforcement Learning*, Advances in Neural Information Processing Systems (NeurIPS), 2023.

[11] Mathematical Components organization, *Mathematical Compenents library*, https://math-comp.github.io/

[12] Mathematical Components team *The SSReflect proof language*, https://rocq-prover.org/doc/V9.2.0/refman/proof-engine/ssreflect-proof-language.html

[13] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu and Guillaume Lample. *Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs*, 2023.

[14] Stanislas Polu and Ilya Sutskever. *Generative language modeling for automated theorem proving.* arXiv:2009.03393, 2020.

[15] Amitayush Thakur, George Tsoukalas, Yeming Wen, Jimmy Xin and Swarat Chaudhuri. *An In-Context Learning Agent for Formal Theorem-Proving*, arXiv:2310.04353, 2024.

[16] Emily First, Markus N. Rabe, Talia Ringer and Yuriy Brun. *Baldur: Whole-Proof Generation and Repair with Large Language Models*, arXiv:2303.04910, 2023.

[17] Laetitia Teodorescu, Guillaume Baudart, Emilio Jesús Gallego Arias, Marc Lelarge. *NLIR: Natural Language Intermediate Representation for Mechanized Theorem Proving.* MathAI@NeuRIPS 2024 - 4th Workshop on Mathematical Reasoning and AI, Dec 2024, Vancouver, Canada. ⟨hal-04886208⟩

[18] *Pytanque*, https://github.com/LLM4Rocq/pytanque

7. Appendix

7.1. Pseudo-code

Algorithm 1 Treefinement

function MAKEBRANCH(node)
  Tactics $\leftarrow$ **arity** different tactic propositions made by the LLM to make progress in node.proof_state, with help of up to **number_examples** examples sampled from GoodSteps
  for tactic $\in$ Tactics **do**
    new_node $\leftarrow$ $\{.parent = \text{node}, .children = \emptyset, .proof_state = \text{typecheck(tactic)}\}$
    node.$children$ $\leftarrow$ node.$children$ $\sqcup$ {new_node}
    Nodes $\leftarrow$ Nodes $\sqcup$ {new_node}
    if new_node.$proof_state$ is an empty goal **then**
      GoodSteps $\leftarrow$ GoodSteps $\sqcup$ RETRIEVEGOODSTEPS(node)
      return True
    else
      insert new_node in PriorityQueue with priority $-\mathbf{penalty}$(new_node)
    end if
  end for
  return False
end function

function MAKETREE(theorem)
  root $\leftarrow$ $\{.parent = None, .children = \emptyset, .proof_state = \text{get_state(theorem)}\}$
  Nodes $\leftarrow$ {root}
  PriorityQueue $\leftarrow$ $\emptyset$
  insert root in PriorityQueue with priority $-\mathbf{penalty}$(root)
  iter_rem $\leftarrow$ **iter_max**
  while iter_rem > 0 **do**
    beam $= \emptyset$
    remove the beam_size elements of PriorityQueue with highest priority and insert them into beam
    iter_rem $\leftarrow$ iter_rem $-$ beam.$size$
    for node $\in$ beam **do**
      if MAKEBRANCH(node) **then**
        return True
      end if
    end for
  end while
  return False
end function

function TREEFINEMENT(dataset)
  GoodSteps $\leftarrow$ InitialExamples
  nb_success $\leftarrow$ 0
  for theorem $\in$ dataset **do**
    if MAKETREE(theorem) **then**
      nb_success $\leftarrow$ nb_success $+$ 1
    end if
  end for
  return nb_success
end function

Algorithm 2 good steps retrievement

```
function DFS(p, node)
    GoodSteps ← GoodSteps ⊔ {(p → node)}
    for v ∈ p.children do
        if v.proof_state is an error then
            DFS(v, node)
        end if
    end for
end function

function RETRIEVEGOODSTEPS(node)
    while node.parent ≠ None do
        p ← node.parent
        while p.proof_state is an error do
            p ← p.parent
        end while
        DFS(p, node)
        node ← p
    end while
end function
```